Autoregressive Modeling, Entropy, and Decoding

11

Chapter Abstract. Large language models are trained with a surprisingly concrete objective: given a prefix, predict the next token. This chapter studies the mathematics behind that objective. A small prompt, `the movie was not very`, runs through the chapter as we define tokens, logits, autoregressive factorization, causal decoder outputs, next-token negative log-likelihood, entropy, cross-entropy, KL divergence, perplexity, and decoding rules. The goal is applied: a reader should be able to compute the loss for a toy language model, explain what perplexity measures, and predict how greedy, beam, temperature, top-k, and nucleus decoding change generated text.

11.1 Running Example and Chapter Roadmap

Modern decoder-style language models are built around a simple question: given the text so far, which token should come next? The model does not begin by choosing a complete paragraph. It repeatedly estimates a probability distribution over the next token, and a decoding rule turns those probabilities into one visible continuation.

The running prompt for this chapter is

`the movie was not very`

with the tiny set of possible next-token choices

$$\mathcal{V} = \{\texttt{good}, \texttt{bad}, \texttt{funny}, \texttt{long}, \texttt{<end>}\}.$$

For this toy set, the model's immediate job is to assign five probabilities:

$$p_\theta(v \mid \texttt{the movie was not very}), \qquad v \in \mathcal{V}.$$

The same idea scales to real systems with tens of thousands of possible tokens.

The first objects are tokens, vocabularies, logits, and normalized probabilities. The probability chain rule then turns a whole text sequence into a product of next-token probabilities. Training and evaluation use negative log-likelihood, cross-entropy, entropy, KL divergence, and perplexity. Generation applies greedy search, beam search, temperature sampling, top-k sampling, nucleus sampling, and prompting to the learned conditional distributions.

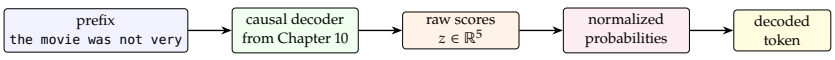

Figure 11.1: Chapter 10 explains the causal decoder computation. This chapter starts at the decoder output and studies the probability model, loss, evaluation metric, and decoding rule.

Embeddings, cross-entropy, sequence models, and causal self-attention supply the machinery. The new object here is the conditional probability distribution that a causal decoder produces at each position [67, 222, 27].

A language model predicts a distribution; decoding chooses one path through that distribution.

Pause and predict

Before seeing any formulas, guess which continuation should receive the larger probability after the prefix `the movie was not very`: `good` or `<end>`. What information from the prefix did you use?

11.2 Tokens, Vocabularies, Logits, and Probabilities

Language models do not directly operate on paragraphs as continuous objects. They operate on discrete tokens. A token may be a word, part of a word, a punctuation mark, or a special marker. Modern systems often use subword tokenization so that a fixed vocabulary can still represent rare words and new combinations [187, 121].

Definition 11.2.1 (Vocabulary and token sequence) *A vocabulary is a finite set*

$$\mathcal{V} = \{v_1, \ldots, v_m\}$$

of allowed tokens. A token sequence of length T is an ordered tuple

$$x_{1:T} = (x_1, \ldots, x_T),$$

where each $x_t \in \mathcal{V}$. A token-ID map is an indexing function $\iota : \mathcal{V} \to \{1, \ldots, m\}$.

The ID of a token is an address, not a measurement. If $\iota(\texttt{good}) = 1$ and $\iota(\texttt{bad}) = 2$, the number $2 - 1$ does not mean that `bad` is one unit away from `good`. The ID is only used to select an embedding row and a coordinate in the output probability vector.

A token ID is an address, not a numerical feature.

Recall from the tensors/data/probability chapter that logits are unconstrained scores and softmax turns them into a probability vector. In a language model, the coordinates of that vector are vocabulary tokens. For vocabulary size m, the decoder returns a logit vector $z \in \mathbb{R}^m$, and we write

$$p_k = \operatorname{softmax}(z)_k = \frac{\exp(z_k)}{\sum_{j=1}^{m} \exp(z_j)}, \qquad k = 1, \ldots, m.$$

Thus p_k is the model's probability for the kth vocabulary token.

For the running prompt, suppose the decoder returns the following logits:

$$z = (2.0, 1.0, 0.0, -0.5, -1.0)$$

for `good`, `bad`, `funny`, `long`, and `<end>`, respectively. The softmax probabilities are approximately:

Token	Logit	exp(logit)	Probability
`good`	2.0	7.389	0.612
`bad`	1.0	2.718	0.225
`funny`	0.0	1.000	0.083
`long`	−0.5	0.607	0.050
`<end>`	−1.0	0.368	0.030

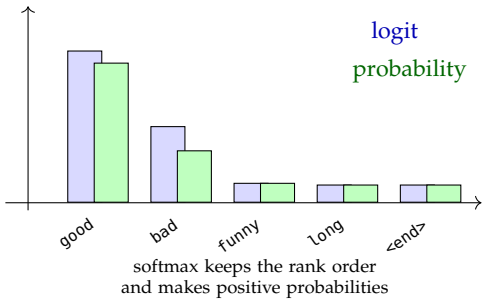

Figure 11.2: Token IDs are addresses, logits are unnormalized scores, and softmax turns those scores into a probability distribution over the vocabulary.

The largest logit gives the largest probability, but the smaller logits still matter. They determine the remaining probability mass and therefore affect sampling-based decoding.

Only relative logit differences matter. Adding the same constant to every logit leaves the softmax probabilities unchanged, the same shift-invariance identity used for stable softmax computation in Section 5.11. Here it means that gaps between token scores, such as the logit for `good` minus the logit for `bad`, determine the next-token distribution, not the absolute logit level.

11.3 Autoregressive Factorization

A language model assigns probabilities to sequences. The key identity is the probability chain rule: a joint probability can be written as a product of conditional probabilities. Autoregressive language modeling uses this identity from left to right.

Definition 11.3.1 (Autoregressive prefix) *For a sequence $x_{1:T}$, the prefix before position t is*

$$x_{<t} = (x_1, \ldots, x_{t-1}).$$

The prefix through position t is

$$x_{\leq t} = (x_1, \ldots, x_t).$$

For $t = 1$, the prefix $x_{<1}$ is empty or contains a special start token.

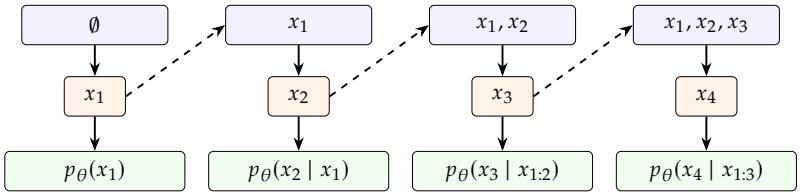

Figure 11.3: The prefix ladder grows one observed token at a time. Each rung contributes one conditional factor to the sequence probability.

Definition 11.3.2 (Autoregressive model) *An autoregressive model represents the probability of a sequence as*

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

The parameter vector θ determines how the model estimates each conditional next-token distribution.

The factorization itself is not a neural-network trick. It is a probability identity. The modeling choice is to approximate the conditional distributions with a neural network, such as a causal transformer decoder [18, 222].

Proposition 11.3.1 (End-token normalized sequence probabilities) *Let $\mathcal{V}$ be a finite token set and let <end> $\notin \mathcal{V}$ be an end token. Suppose that for every finite prefix $u \in \mathcal{V}^*$, the model defines a normalized next-token distribution*

$$\sum_{v \in \mathcal{V} \cup \{\texttt{<end>}\}} p_\theta(v \mid u) = 1,$$

and suppose generation stops the first time <end> is emitted. Then the probability assigned to a finite sequence $x_{1:T} \in \mathcal{V}^T$ that ends at the next step is

$$p_\theta(x_{1:T}, \texttt{<end>}) = \left(\prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}) \right) p_\theta(\texttt{<end>} \mid x_{1:T}).$$

If the process emits <end> with probability one eventually, these end-token-terminated sequence probabilities sum to one over all finite sequences.

Proof. The displayed probability is the probability of drawing $x_1, \ldots, x_T$ and then drawing <end> from the next-token distributions. The events "first stop after finite sequence $x_{1:T}$" are disjoint for different finite sequences. Their total probability is the probability that the sequential process eventually stops. Under the stated assumption, that probability is one. □

The end token is what turns local next-token distributions into a proper distribution over finite strings. If the model can continue forever with positive probability, some mass remains on infinite continuations instead of on completed sequences.

For an autoregressive model with positive next-token probabilities, taking logs of the product in the autoregressive factorization gives the sequence

log-likelihood

$$\log p_\theta(x_{1:T}) = \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

For example, suppose a three-token sequence has conditional probabilities

$$p(x_1) = 0.10, \qquad p(x_2 \mid x_1) = 0.40, \qquad p(x_3 \mid x_{1:2}) = 0.50.$$

Then

$$p(x_{1:3}) = 0.10 \cdot 0.40 \cdot 0.50 = 0.020.$$

Equivalently,

$$\log p(x_{1:3}) = \log(0.10) + \log(0.40) + \log(0.50).$$

Definition 11.3.3 (Average token negative log-likelihood) *For one sequence $x_{1:T}$, the average token negative log-likelihood is*

$$\mathrm{NLL}(x_{1:T}; \theta) = -\frac{1}{T} \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

This quantity is a loss. It is small when the model assigns high probability to the observed tokens and large when it assigns low probability. The average, rather than the raw sum, allows comparison of sequences with different lengths more sensibly.

Products become sums after logs; sums become averages for reporting losses.

Pause and predict

Two candidate continuations have probabilities $0.6 \cdot 0.2$ and $0.3 \cdot 0.5$. Which has the larger sequence probability? Which has the larger sum of log-probabilities?

11.4 Causal Decoder Outputs

A causal decoder reads a token sequence and produces a vector at each position. Chapter 10 develops the attention algebra. Here we only need the interface between that decoder and the language-model probability distribution.

Definition 11.4.1 (Context window) *The context window length C is the maximum number of token positions the model can condition on in one forward pass. If a document is longer than C, training and generation must split or slide across the document.*

Let a training window contain $T \le C$ token positions. After embedding lookup and causal transformer blocks, the decoder returns hidden vectors

$$h_1, \ldots, h_T \in \mathbb{R}^d.$$

Stacking them gives a matrix $H \in \mathbb{R}^{T \times d}$. Causal masking means that h_t can depend on tokens through position t, but not on future tokens $x_{t+1}, x_{t+2}, \ldots$ [222].

In the notation of the attention and transformer algebra chapter, one can read the sequence computation as

$$H = \text{Decoder}_\theta(E[x_{1:T}] + P_{1:T}; M_{\text{causal}}) \in \mathbb{R}^{T \times d},$$

where $E[x_{1:T}]$ is the matrix of token embeddings, $P_{1:T}$ contains positional information, and M_{causal} is the lower-triangular causal attention mask. The prefix-function property of causal transformer rows is exactly what makes row $H_{t,:}$ legal for predicting the next token x_{t+1}.

Definition 11.4.2 (Output projection) *For vocabulary size m and hidden dimension d, an output projection, also called an unembedding matrix, is a matrix*

$$W_U \in \mathbb{R}^{m \times d}$$

with bias $b \in \mathbb{R}^m$. The next-token logits at position t are

$$z_t = W_U h_t + b.$$

Equivalently, if $w_{U,k}^\top$ is row k of W_U, then

$$z_{t,k} = w_{U,k}^\top h_t + b_k.$$

Thus the logit for token v_k is a dot-product score between the current hidden state and an output-token row, plus a bias. This reuses the embedding geometry from the embeddings chapter: dot products and angles are not just input-side tools, but also appear in the output scores before softmax.

Applying the same output projection to every row gives the full logit matrix

$$Z = HW_U^\top + \mathbf{1}_T b^\top \in \mathbb{R}^{T \times m},$$

where $\mathbf{1}_T$ is a length-T column vector of ones. Row $Z_{t,:}$ becomes the vocabulary logit vector used for the shifted next-token label at position t.

Definition 11.4.3 (Decoder next-token distribution) *The model distribution for the next token after prefix $x_{\le t}$ is*

$$p_\theta(x_{t+1} = v_k \mid x_{\le t}) = \text{softmax}(z_t)_k, \qquad k = 1, \ldots, m.$$

For a finite training sequence $x_{1:T}$, this shifted label exists for $t = 1, \ldots, T-1$. If an explicit end token is appended, the final input position can instead predict $x_{T+1} =$ <end>.

The output projection is often large. If $m = 50{,}000$ and $d = 4096$, then W_U has $50{,}000 \cdot 4096 = 204{,}800{,}000$ entries before counting the bias. This is one reason that vocabulary size and tokenization are practical design choices, not only preprocessing details.

The output layer scales with vocabulary size times hidden dimension.

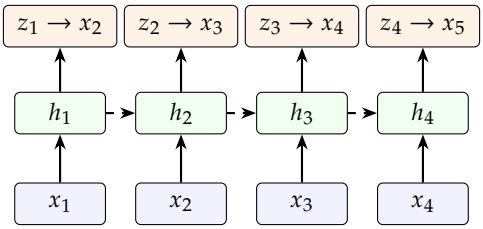

Figure 11.4: A causal decoder produces one hidden vector per position. The logits from position t are trained to predict the next token x_{t+1}.

11.5 Next-Token Training Objective

Language-model pretraining turns raw token sequences into supervised examples. At each position, the input is the prefix and the label is the next token. No human-written class label is needed.

Definition 11.5.1 (Shifted next-token labels) *For a sequence $x_{1:T}$, predictions from positions $t = 1, \ldots, T-1$ are compared with labels x_{t+1}. If training appends an explicit end token, then $x_{T+1} =$ <end> can also be used as the shifted label for position T. Thus the input and label arrays are shifted by one position, with the valid positions determined by the available shifted labels.*

For the text `the movie was not very good`, the shifted training pairs are:

Prefix available to the model	Next-token label
`the`	`movie`
`the movie`	`was`
`the movie was`	`not`
`the movie was not`	`very`
`the movie was not very`	`good`

Definition 11.5.2 (Teacher forcing) *Teacher forcing means that during training, the model conditions on the true previous tokens from the dataset, not on tokens it sampled earlier.*

Teacher forcing is what makes the loss parallelizable over positions. A single forward pass through a causal decoder can produce losses for many next-token labels. During generation, this changes: the model conditions on tokens that the decoding rule already selected.

Definition 11.5.3 (Masked next-token loss) *Let $M_{i,t} \in \{0, 1\}$ indicate whether example i, position t, is a real next-token target rather than padding. At that position, the model reads the prefix $x_{i,\le t}$ and predicts the shifted label $x_{i,t+1}$. For a dataset of token sequences, the masked average next-token loss is*

$$\mathcal{L}(\theta) = -\frac{1}{\sum_{i,t} M_{i,t}} \sum_{i,t} M_{i,t} \log p_\theta(x_{i,t+1} \mid x_{i,\le t}).$$

The sums include only positions for which the shifted label $x_{i,t+1}$ is defined; the mask removes padding and any unavailable target positions.

Table 11.1: Training aligns each input position with the next-token label. Padding fills the rectangular minibatch, but the mask removes padded labels from the loss.

Quantity	Position 1	Position 2	Position 3	Position 4
Input row A	`the`	`movie`	`was`	`<pad>`
Shifted label A	`movie`	`was`	`<end>`	`<pad>`
Loss mask A	1	1	1	0
Input row B	`not`	`very`	`<pad>`	`<pad>`
Shifted label B	`very`	`<end>`	`<pad>`	`<pad>`
Loss mask B	1	1	0	0

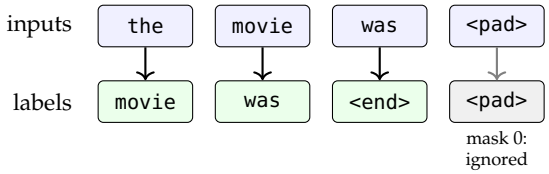

Figure 11.5: Next-token training shifts labels one position to the left. Real shifted labels contribute to the loss; padding positions are present only to make a rectangular minibatch.

Padding is storage. The mask defines which tokens count in the loss.

The mask is a mathematical statement about the objective. Padding may be needed to make equal-size arrays in a minibatch, but padding tokens should not contribute to the empirical risk.

Example 11.5.1 (Masked average loss) Suppose two padded sequences produce token losses

$$(0.2, 0.7, 1.1, 0.0) \quad \text{and} \quad (0.5, 0.4, 0.0, 0.0),$$

with masks

$$(1, 1, 1, 0) \quad \text{and} \quad (1, 1, 0, 0).$$

The masked average loss is

$$\frac{0.2 + 0.7 + 1.1 + 0.5 + 0.4}{5} = 0.58.$$

The zeros at padded positions are ignored because their mask entries are zero.

Equivalently, for any factor whose observed target token is $x_t = v_y$, the one-hot target e_y selects the corresponding negative log-probability:

$$-\sum_{k=1}^{m} (e_y)_k \log p_\theta(v_k \mid x_{<t}) = -\log p_\theta(v_y \mid x_{<t}).$$

All terms except the true-token term vanish because the one-hot vector has a 1 in coordinate y and 0 elsewhere.

At the population level, the same loss is conditional cross-entropy. The next identity is $H(P, q_\theta) = H(P) + D_{\mathrm{KL}}(P \,\|\, q_\theta)$ averaged over prefixes, assuming $q_\theta(v \mid C) > 0$ wherever $P(v \mid C) > 0$.

Proposition 11.5.1 (Population next-token NLL decomposition) *Let $C = X_{<t}$ be a random prefix, let $P(\cdot \mid C)$ be the true next-token conditional distribution, and let $q_\theta(\cdot \mid C)$ be the model distribution. The expected teacher-forced negative log-likelihood is*

$$\mathbb{E}_C \mathbb{E}_{Y \sim P(\cdot \mid C)}[-\log q_\theta(Y \mid C)] = \mathbb{E}_C H(P(\cdot \mid C)) + \mathbb{E}_C D_{\mathrm{KL}}(P(\cdot \mid C) \,\|\, q_\theta(\cdot \mid C)).$$

The entropy term is fixed by the data distribution and cannot be reduced by the model. The KL term is minimized when the model conditional distribution matches the true next-token conditional distribution, so maximum likelihood is statistically targeting the full conditional distribution, not only the most likely next token.

Recall the softmax-cross-entropy gradient from the optimization chapter: for logits $z \in \mathbb{R}^m$, probabilities $p = \mathrm{softmax}(z)$, and one-hot target e_y,

$$\ell(z, y) = -\sum_{k=1}^{m} (e_y)_k \log p_k = -\log p_y,$$

has logit gradient

$$\nabla_z \ell(z, y) = p - e_y.$$

The language-modeling specialization is positional: at each valid masked position (i, t), the target class is the shifted next token $y = x_{i,t+1}$, and the same gradient applies to that position's vocabulary logits. This is the same gradient shape as ordinary multiclass classification, but with a different supervised label at every valid token position.

The applied reason this loss matters is direct: the model is trained to put high probability on the next token that actually appeared in the training text [67, 18, 168].

11.6 Entropy, Cross-Entropy, KL Divergence, and Perplexity

The next-token distribution is useful only if we can reason about its uncertainty and score its predictions. The tensors/data/probability chapter introduced entropy, cross-entropy, KL divergence, and the decomposition $H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$. Here we only specialize those quantities to vocabulary distributions and use them to define the new language-model metric: perplexity [189, 41, 21].

For a probability distribution p over m tokens, the recalled entropy is

$$H(p) = -\sum_{k=1}^{m} p_k \log p_k,$$

with terms satisfying $p_k = 0$ treated as 0. With natural logarithms, entropy is measured in nats.

Entropy is high when probability mass is spread across many tokens and low when one token dominates. For this chapter's movie-review prefix, a model may be uncertain between `good` and `bad`. For a more constrained factual prefix, a good model should usually be much less uncertain.

For token distributions, the general cross-entropy and KL divergence formulas specialize to sums over the vocabulary. For target distribution p and model distribution q over the same vocabulary, the cross-entropy is

$$H(p, q) = -\sum_{k=1}^{m} p_k \log q_k.$$

The KL divergence from p to q is

$$D_{\mathrm{KL}}(p \| q) = \sum_{k=1}^{m} p_k \log \frac{p_k}{q_k}.$$

As recalled from that earlier probability discussion, when $q_k > 0$ whenever $p_k > 0$, these quantities satisfy

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q).$$

For one-hot next-token training, the target distribution has all mass on the observed token. In that case, the token loss is just the negative log probability assigned to the observed token.

Definition 11.6.1 (Perplexity) *For an average token negative log-likelihood L, the perplexity is*

$$\mathrm{PPL} = \exp(L).$$

When logarithms use base 2, perplexity is 2^L instead.

Perplexity is an exponentiated loss. If the average negative log-likelihoods are 0.7, 1.5, and 3.0, the corresponding perplexities are approximately

$$e^{0.7} \approx 2.01, \qquad e^{1.5} \approx 4.48, \qquad e^{3.0} \approx 20.09.$$

Lower perplexity usually means better held-out next-token prediction under the same evaluation setup.

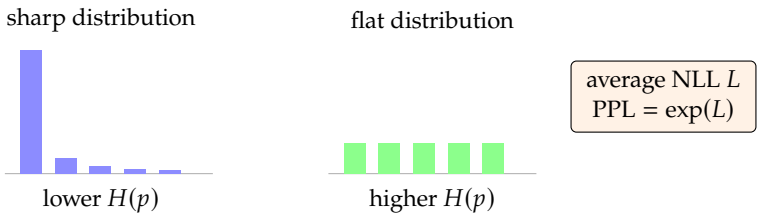

Figure 11.6: Entropy describes how spread out a next-token distribution is. Perplexity exponentiates the average next-token negative log-likelihood.

Example 11.6.1 (Two uncertainty patterns) The distribution

$$p = (0.80, 0.10, 0.05, 0.03, 0.02)$$

is sharper than

$$r = (0.20, 0.20, 0.20, 0.20, 0.20).$$

The sharp distribution has lower entropy because the model strongly favors one token. The uniform distribution has entropy $\log 5$, the maximum possible entropy over five tokens.

Perplexity measures held-out token prediction, not factuality or usefulness. A model can give high probability to a fluent false sentence, especially when the training distribution contains similar false or ungrounded text.

Perplexity scores prediction, not truth.

Pause and predict

Suppose two models have the same top predicted token, but one assigns probability 0.90 to it while the other assigns probability 0.35. Which model has lower loss if that token is correct? Which model is more uncertain?

11.7 From Training to Generation

Training and generation use the same conditional distributions in different ways. Training scores true text from the dataset. Generation creates new text by feeding selected tokens back into the prefix.

Definition 11.7.1 (Autoregressive generation loop) *Given an initial prompt $x_{1:t}$, generation repeats the following steps:*

1. *Compute $p_\theta(\cdot \mid x_{\le t})$.*
2. *Use a decoding rule to choose or sample $\hat{x}_{t+1}$.*
3. *Append $\hat{x}_{t+1}$ to the context.*
4. *Stop at a special end token or a maximum length.*

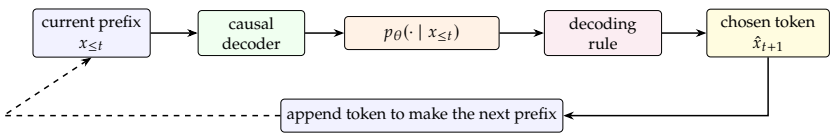

Figure 11.7: Generation feeds the selected token back into the prefix before the next probability distribution is computed.

The generated sequence is random if the decoding rule samples from a distribution. It is deterministic if the decoding rule always chooses the same token from the same probability vector.

Example 11.7.1 (A choice changes context) After a prompt ending in "not very," suppose a model assigns probability 0.61 to `good` and 0.23 to `bad`. The next context depends on the chosen token:

choose `good` `the movie was not very good`
choose `bad` `the movie was not very bad.`

The next probability table can be different because the prefix is different.

The practical issue is feedback. During training, the model sees true prefixes from the dataset. During generation, it may see prefixes partly produced by itself. Early choices can make later choices easier, harder, more repetitive, or more surprising, so the decoding rule is part of the visible behavior.

Training scores true text; generation feeds back chosen text.

Pause and predict

After the model chooses `bad` instead of `good`, predict one way the next-token distribution for punctuation or an ending could change. Why might a teacher-forced validation loss miss this rollout effect?

Remark 11.7.1 (Exposure to self-generated context) The phrase exposure bias is often used for the gap between training on true prefixes and generating from model-produced prefixes. For this chapter, the main applied lesson is enough: evaluate generated text under the decoding rule you plan to use, not only by the training loss [16].

The next two sections separate deterministic search rules from sampling rules. Both begin with the same model probabilities. They differ in how they use those probabilities to produce text.

11.8 Deterministic Decoding

Deterministic decoding rules return the same continuation whenever the model, prompt, and settings are the same. The two simplest examples are greedy decoding and beam search.

Definition 11.8.1 (Greedy decoding) *Greedy decoding selects the highest-probability token at each step:*

$$\hat{x}_{t+1} = \arg\max_{v \in \mathcal{V}} p_\theta(v \mid \hat{x}_{\le t}).$$

Greedy decoding is easy to understand and cheap to run. Its weakness is also easy to understand: a locally best first token may lead to a worse full sequence.

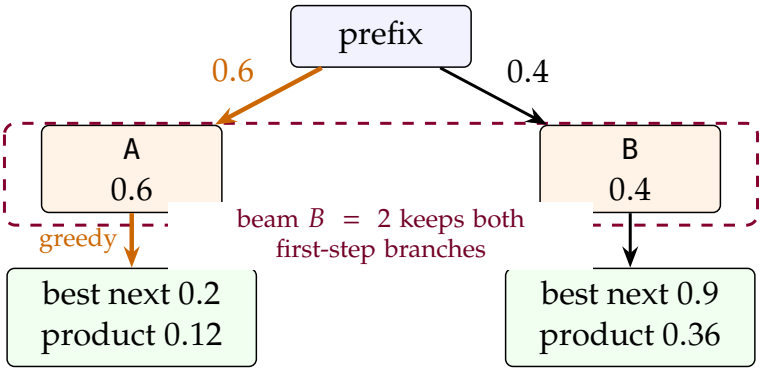

Figure 11.8: Greedy decoding commits to the locally best branch. Beam search keeps several partial branches, so a lower-probability first token can survive long enough to produce a better sequence score.

Example 11.8.1 (Greedy can miss a better two-token sequence) Suppose the first step has

$$p(\texttt{A}) = 0.6, \qquad p(\texttt{B}) = 0.4.$$

After `A`, suppose the best second token is `a`, with probability 0.2. After `B`, suppose the best second token is `b`, with probability 0.9. Greedy chooses `A` first and gets sequence probability $0.6 \cdot 0.2 = 0.12$. The alternative path (`B`, `b`) has probability $0.4 \cdot 0.9 = 0.36$.

Beam search matters when one early locally plausible choice can make later choices much better or worse. It is most useful when there is a relatively clear target string, as in some translation or transcription settings [205]. Instead of committing to one prefix immediately, beam search pays extra computation to keep several partial continuations alive. Greedy decoding is the special case with beam width $B = 1$.

Example 11.8.2 (Beam search keeps partial sequences) In the same toy tree, a beam with width $B = 2$ keeps two partial sequences after each pruning step:

Step	Candidate prefixes scored	Beam after pruning
0	empty prefix	{()}
1	(`A`) with score log(0.6); (`B`) with score log(0.4)	{(`A`), (`B`)}
2	(`A`, `a`) with score log(0.6 · 0.2); (`B`, `b`) with score log(0.4 · 0.9)	{(`B`, `b`), (`A`, `a`)}

The point is not that `B` was best at the first step. It was not. The point is that the beam kept `B` alive long enough for its better second step to be compared with the greedy branch.

Definition 11.8.2 (Beam search) *Fix a beam width B and a score S for partial sequences. Beam search starts with $\mathcal{B}_0 = \{()\}$. Given the beam $\mathcal{B}_{t-1}$, it forms candidate extensions*

$$\mathcal{C}_t = \{(\hat{x}_{1:t-1}, v) : \hat{x}_{1:t-1} \in \mathcal{B}_{t-1},\ v \in \mathcal{V}\}$$

and sets $\mathcal{B}_t$ to the at most B candidates in $\mathcal{C}_t$ with largest score. With log-probability scoring,

$$S(\hat{x}_{1:t}) = \sum_{s=1}^{t} \log p_\theta(\hat{x}_s \mid \hat{x}_{<s}).$$

This definition expands every vocabulary token for clarity. Implementations often restrict the expansion to likely next tokens, keep completed sequences that have emitted an end token, and return the highest-scoring completed sequence at the end of the search. Increasing B explores more alternatives but increases computation. In open-ended generation, maximizing probability too aggressively can produce text that is dull or repetitive. High likelihood is a training objective, but it is not always the best decoding objective for creative or conversational text [90].

Definition 11.8.3 (Length-normalized score) *A simple length-normalized beam score is*

$$S_{\text{avg}}(\hat{x}_{1:t}) = \frac{1}{t} \sum_{s=1}^{t} \log p_\theta(\hat{x}_s \mid \hat{x}_{<s}).$$

Length normalization matters because raw sequence probabilities shrink as more probabilities are multiplied. Without a length adjustment, a search rule can prefer short outputs for a mathematical reason unrelated to answer quality.

Beam scores need length care because products shrink with every token.

Pause and predict

Candidate A has length 3 and average log-probability -0.20. Candidate B has length 8 and average log-probability -0.30. Predict which candidate wins under average log-probability and why the raw product would penalize B even more strongly.

11.9 Sampling-Based Decoding

Sampling-based decoding treats the model output as a distribution to sample from. These methods are common when there are many acceptable continuations. They trade determinism for diversity.

Figure 11.9: Decoding controls act after the model has produced next-token probabilities. Temperature changes the concentration of the distribution, top-k keeps a fixed number of high-probability candidates, and top-p keeps an adaptive set whose cumulative mass crosses the chosen threshold.

Definition 11.9.1 (Temperature-scaled softmax) *For temperature $\tau > 0$, define*

$$p_k(\tau) = \frac{\exp(z_k/\tau)}{\sum_{j=1}^{m} \exp(z_j/\tau)}.$$

Lower τ sharpens the distribution; higher τ flattens it.

With the running logits $z = (2, 1, 0, -0.5, -1)$, lowering the temperature below 1 makes `good` even more dominant. Raising the temperature spreads probability mass toward `bad`, `funny`, and the other lower-scoring tokens.

τ	`good`	`bad`	`funny`	`long`	`<end>`	$H(p(\tau))$
0.5	0.860	0.116	0.016	0.006	0.002	0.488
1.0	0.612	0.225	0.083	0.050	0.030	1.099
2.0	0.403	0.244	0.148	0.115	0.090	1.459

The entropy column makes the applied effect visible: higher temperature makes the next-token distribution less concentrated and therefore more uncertain.

Definition 11.9.2 (Top-k sampling) *Top-k sampling keeps only the k highest-probability tokens, sets all other probabilities to zero, renormalizes the remaining probabilities to sum to one, and samples from the result.*

Definition 11.9.3 (Nucleus, or top-p, sampling) *Nucleus sampling sorts tokens by decreasing probability and keeps the smallest set whose cumulative probability is at least p. It then renormalizes over that set and samples from the result.*

Top-k keeps a fixed number of candidates. Top-p keeps a variable number of candidates depending on how concentrated the distribution is. This is useful because a confident distribution may need only a few reasonable choices, while an uncertain distribution may need more.

Table 11.2: Top-k cuts after a fixed number of sorted tokens. Top-p cuts after the cumulative mass first reaches the threshold.

Sorted token	Probability	Cumulative mass	Top-2	Top-$p = 0.90$
`good`	0.61	0.61	keep	keep
`bad`	0.23	0.84	keep	keep
`funny`	0.08	0.92	drop	keep
`long`	0.05	0.97	drop	drop
`<end>`	0.03	1.00	drop	drop

Example 11.9.1 (Top-k and top-p candidate sets) For probabilities

$$(0.61, 0.23, 0.08, 0.05, 0.03),$$

top-2 keeps `good` and `bad`. Top-p with $p = 0.90$ keeps `good`, `bad`, and `funny`, because $0.61 + 0.23 = 0.84 < 0.90$, while $0.61 + 0.23 + 0.08 = 0.92$.

Decoding rule	Candidate set for the example	Applied consequence
Greedy	{`good`}	deterministic and narrow
Temperature sampling	all five tokens	changes probabilities, not support
Top-2	{`good`, `bad`}	fixed-size support
Top-$p = 0.90$	{`good`, `bad`, `funny`}	adaptive support

Sampling settings change generation behavior without changing model weights.

Sampling settings are applied after training. They do not change the model parameters. They change how probability mass is converted into generated text. Holtzman et al. argue that maximization-based decoding can lead to neural text degeneration and motivate nucleus sampling for open-ended text [90]; top-k sampling is also widely used in neural generation systems [61].

Pause and predict

If a next-token distribution is nearly uniform over many tokens, will top-p with $p = 0.90$ keep many tokens or few tokens? What if the largest token already has probability 0.92?

11.10 Prompting and In-Context Conditioning

Prompting is conditioning. If c is the prompt and $y_{1:U}$ is a possible answer, the model assigns

$$p_\theta(y_{1:U} \mid c) = \prod_{j=1}^{U} p_\theta(y_j \mid c, y_{1:j-1}).$$

Changing c changes the conditional distribution. The parameter vector θ is unchanged.

Definition 11.10.1 (Prompt and in-context example) *A prompt is the finite token sequence $c_{1:L} \in \mathcal{V}^L$ supplied before generation begins. An in-context example is a designated subsequence of the prompt that encodes an input-output pair (u_i, v_i) before the query tokens.*

For fixed parameters θ, two prompts c and c' can assign different probabilities to the same answer $y_{1:U}$. Their log-probability difference is

$$\Delta(c, c'; y_{1:U}) = \log p_\theta(y_{1:U} \mid c') - \log p_\theta(y_{1:U} \mid c).$$

This quantity can be nonzero even though θ is identical in both terms. Both probabilities are evaluated by the same parameterized model p_θ, but the conditioning prefix differs. Since the conditional distribution of the next token is a function of the prefix, changing the prefix can change every factor in the product above without changing any parameter.

For example, a tiny sentiment prompt might be

```
Review: the movie was wonderful      Sentiment: positive
Review: the movie was boring         Sentiment: negative
Review: the movie was not very good  Sentiment:
```

The demonstrations are not gradient updates. They are tokens in the context window. They can still strongly affect the next-token distribution because the model conditions on them.

Example 11.10.1 (A two-answer logit calculation) Suppose the next token must be either `positive` or `negative`. With a vague prompt, a model might produce logits $(0.2, 0.1)$, giving

$$p_\theta(\texttt{positive} \mid c) = \frac{e^{0.2}}{e^{0.2} + e^{0.1}} \approx 0.525.$$

After adding demonstrations to the prompt, the same parameters might produce logits $(1.4, -0.6)$, giving

$$p_\theta(\texttt{positive} \mid c') = \frac{e^{1.4}}{e^{1.4} + e^{-0.6}} \approx 0.881.$$

The calculation changed because the conditioning context changed, not because a training step occurred.

Definition 11.10.2 (Instruction and query) *For a prompt decomposed as $c = \mathrm{encode}(I, E, q)$, the instruction I is the token subsequence that specifies the task rule or output format, E is the optional collection of in-context examples, and the query q is the token subsequence whose answer is to be generated. All three are part of the conditioning context.*

Prompt design changes the information available to the conditional distribution. A vague prompt may leave many likely answer formats. A prompt with examples and an explicit output format can concentrate probability mass on the desired form. Brown et al. studied this few-shot setting for

A prompt changes conditioning, not parameters.

large autoregressive language models, where examples are provided as text rather than by changing model parameters [27].

Remark 11.10.1 (Prompting is not the same as learning) In-context examples can make a model imitate a pattern in the prompt. That is different from proving that the model has learned a reliable concept or that it will behave correctly outside the prompt distribution.

This chapter stops at conditioning. Preference learning, reinforcement learning, and other post-training methods are different mathematical operations: they change θ, a reward model, or a policy used to sample answers.

11.11 Evaluation, Calibration, and Failure Modes

The most direct evaluation for a pretrained language model is held-out negative log-likelihood. The model is trained on one text collection and evaluated on separate validation or test text. This separation matters because memorizing training text is not the same as predicting unseen text.

Definition 11.11.1 (Held-out validation loss) *Let $\mathcal{D}_{\text{val}}$ be a held-out set of token sequences, and let $m_t^{(n)} \in \{0,1\}$ indicate which next-token positions are included in the loss. Included positions must have a defined shifted label. The held-out validation loss is*

$$\widehat{L}_{\text{val}}(\theta) = \frac{1}{N_{\text{val}}} \sum_n \sum_t m_t^{(n)} \left[-\log p_\theta\left(x_{t+1}^{(n)} \mid x_{1:t}^{(n)}\right)\right],$$

$$N_{\text{val}} = \sum_n \sum_t m_t^{(n)}.$$

Validation loss and perplexity are only comparable when the evaluation setup is compatible. Tokenization, context length, text preprocessing, and loss units all matter. A model evaluated with word tokens and a model evaluated with subword tokens may produce perplexities that should not be compared as raw numbers. With natural logarithms, the reported token perplexity is $\exp(\widehat{L}_{\text{val}})$.

Suppose validation sequences are sampled independently from a distribution q_{val} and were not used for parameter updates or model selection. Because $\widehat{L}_{\text{val}}$ weights every included token equally, it estimates the token-weighted population quantity

$$L_{q_{\text{val}}}^{\text{tok}}(\theta) = \frac{\mathbb{E}_{X\sim q_{\text{val}}}\left[\sum_t m_t(X)\left[-\log p_\theta(X_{t+1} \mid X_{1:t})\right]\right]}{\mathbb{E}_{X\sim q_{\text{val}}}\left[\sum_t m_t(X)\right]},$$

not the loss on an arbitrary deployment distribution. For a fixed evaluation rule, each included token contributes a random negative-log-likelihood term drawn under q_{val}. Averaging those terms gives an empirical mean for the corresponding expectation. If deployment examples follow another

Prediction bin	Mean prob.	Empirical freq.	Reading
0.20–0.40	0.31	0.29	close
0.40–0.60	0.52	0.41	overconfident
0.60–0.80	0.71	0.62	overconfident
0.80–1.00	0.88	0.86	close

Table 11.3: An illustrative calibration table bins model probabilities and then checks how often the corresponding events occurred. Perfect calibration would make the last two numeric columns match in each bin.

distribution, the expectation being estimated changes. A per-sequence average would be a different metric and should be reported separately.

Definition 11.11.2 (Calibration) *A top-token next-token predictor is calibrated when its confidence random variable equals its conditional correctness probability. Let*

$$\widehat{Y} = \arg\max_{v} p_\theta(v \mid X_{1:t}),$$

$$C = \max_{v} p_\theta(v \mid X_{1:t}),$$

$$Z = \mathbf{1}\{X_{t+1} = \widehat{Y}\}.$$

Exact calibration means

$$\mathbb{E}[Z \mid C] = C$$

almost surely.

Calibration is different from accuracy. A model can often choose the right top token while being overconfident or underconfident. Calibration is important when probabilities are used to decide whether to defer, ask for more context, or choose a conservative decoding setting [75].

With confidence bins $B_1, \ldots, B_K$, a common summary is expected calibration error:

$$\mathrm{ECE} = \sum_{k=1}^{K} \frac{|B_k|}{N} \left|\mathrm{acc}(B_k) - \mathrm{conf}(B_k)\right|,$$

where $\mathrm{acc}(B_k)$ is the empirical correctness rate in bin k, and $\mathrm{conf}(B_k)$ is the mean predicted confidence in that bin.

Definition 11.11.3 (Distribution shift) *Distribution shift occurs when the deployment distribution differs from the validation distribution: $q_{\mathrm{deploy}} \neq q_{\mathrm{val}}$. In that case, $\widehat{L}_{\mathrm{val}}$ estimates $L_{q_{\mathrm{val}}}(\theta)$, while deployment quality depends on $L_{q_{\mathrm{deploy}}}(\theta)$.*

Distribution shift is common in applied deep learning. A model trained mostly on web text may face medical notes, legal contracts, source code, classroom dialogue, or private organizational documents. Good held-out perplexity on one source does not guarantee good behavior on another.

Definition 11.11.4 (Memorization, contamination, and hallucination) *Fix a training corpus $\mathcal{T}$, an evaluation set $\mathcal{E}$, a near-duplicate relation $\sim$, and an external verifier V for factual claims. Memorization is reproduction of a training substring from $\mathcal{T}$ above a chosen length or similarity threshold. Data contamination occurs when some evaluation example $e \in \mathcal{E}$ has a training example $t \in \mathcal{T}$ with $e \sim t$. A hallucination is a generated factual claim a for which the verifier returns $V(a) = 0$, even though the text presents a as true.*

The mathematics has not failed when these failure modes appear; the chosen quantity is measuring something narrower. Negative log-likelihood measures predictive fit to observed tokens under the evaluation distribution. It does not verify external facts, guarantee safe advice, or prove that an answer satisfies a user's intention. Reports on GPT-style models emphasize that large language models can be fluent while still making errors or reflecting undesirable patterns in the training data [168, 27].

Use the metric that matches the deployment risk.

Pause and predict

Two models have the same validation perplexity. One was evaluated on text from the same source as deployment; the other was evaluated on unrelated text. Which comparison is more useful for choosing a deployed model, and why?

11.12 Summary

An autoregressive language model estimates next-token conditional probabilities. The probability chain rule writes a sequence probability as $p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t})$, and the logarithm turns this product into a sum of token log-probabilities.

A causal decoder produces hidden vectors, an output projection turns those hidden vectors into logits, and the softmax turns logits into probability vectors over the vocabulary. During training, shifted labels and teacher forcing let one forward pass create many next-token losses. Padding masks keep non-real positions out of the objective.

Entropy measures uncertainty, cross-entropy measures average surprise under a model distribution, KL divergence measures the extra cost from using the wrong distribution, and perplexity exponentiates average token negative log-likelihood. These quantities are central to language-model evaluation, but they do not by themselves measure factuality, safety, or usefulness.

Generation uses the trained conditional distributions one step at a time. Greedy decoding, beam search, temperature sampling, top-k sampling, and nucleus sampling are different decision rules applied to the same kind of probability vector. Prompting changes the conditioning context, not the model parameters.

From next-token prediction, the next question is decision-making from feedback. Chapter 12 introduces Markov decision processes and reinforcement learning foundations. Chapter 13 then revisits language models as systems optimized for preferences and task-level behavior, not just next-token prediction.

11.13 Exercises

Work through these eleven autoregressive modeling exercises. When a prompt asks for a calculation, include enough intermediate values to make the answer checkable.

Exercise 1. For the sequence `the movie was not very good`, list the prefixes used to predict `movie`, `not`, and `good`. If a causal decoder returns $H \in \mathbb{R}^{5\times d}$ for the five input positions and $W_U \in \mathbb{R}^{m\times d}$, what is the shape of $Z = HW_U^\top + \mathbf{1}_5 b^\top$? Which row of Z is compared with the shifted label `good`?

Exercise 2. For logits $z = (1, 0, -1)$, compute the softmax probabilities to three decimal places. Then add 5 to every logit and verify that the probabilities do not change.

Exercise 3. A three-token sequence has conditional probabilities 0.20, 0.50, and 0.40. Compute the sequence probability and the sequence log-probability using natural logarithms.

Exercise 4. A model assigns probability 0.05 to the observed next token. Compute the next-token negative log-likelihood. Then compare it with the loss when the observed token probability is 0.80.

Exercise 5. Two padded next-token training examples have token losses $(0.3, 0.9, 0.0)$ and $(0.4, 0.2, 1.1)$, with shifted-label masks $(1, 1, 0)$ and $(1, 1, 1)$. Compute the masked average loss. How many shifted next-token labels contribute to the denominator?

Exercise 6. Compute the entropy of $p = (1/2, 1/2)$ and $q = (3/4, 1/4)$ using natural logarithms. Which distribution is more uncertain?

Exercise 7. If a model has average token negative log-likelihood $L = 1.2$, compute its perplexity. What does this number measure, and what does it not measure?

Exercise 8. A two-step decoding tree has $p(\texttt{A}) = 0.6$, $p(\texttt{B}) = 0.4$, $p(\texttt{a} \mid \texttt{A}) = 0.2$, and $p(\texttt{b} \mid \texttt{B}) = 0.9$. Which first token does greedy decoding choose? Compute the two completed log scores $\log p(\texttt{A}, \texttt{a})$ and $\log p(\texttt{B}, \texttt{b})$. Which completed path survives as best if a width-2 beam keeps both first-step prefixes?

Exercise 9. For sorted probabilities $(0.50, 0.20, 0.15, 0.10, 0.05)$, compute the renormalized distribution used by top-2 sampling. Then compute the renormalized distribution used by nucleus sampling with threshold $p = 0.80$.

Exercise 10. Explain in two or three sentences why a low held-out perplexity does not guarantee factual correctness, instruction following, or safe deployment behavior. Name one prompting or in-context-conditioning diagnostic that would reveal a failure not visible from perplexity alone.

Exercise 11. Starting from the definitions of entropy, cross-entropy, and KL divergence, derive $H(p, q) = H(p) + D_{\mathrm{KL}}(p\|q)$. Why does this mean that minimizing cross-entropy over q also minimizes $D_{\mathrm{KL}}(p\|q)$ when p is fixed?

11.14 Homework: Symbolic Next-Token Modeling

This homework uses SymPy to make next-token loss and decoding calculations concrete. The first part uses three candidate next tokens, three symbolic logits, one observed target token, and a temperature parameter. Later parts add a tiny shifted-token batch, filtered sampling distributions, and beam-style log scores. No deep-learning library is required [145].

Submission model. Submit the completed starter script, the main hand derivation, the SymPy output needed to verify it, and a short interpretation. External computer-algebra tools may be used only as optional sanity checks.

Mathematical setup

Let z_1, z_2, z_3 be logits for the candidate next tokens `good`, `bad`, and `long`. Define

$$p_k = \frac{\exp(z_k)}{\exp(z_1) + \exp(z_2) + \exp(z_3)}.$$

Assume the observed next token is the first token, `good`, so the one-hot target is $e_1 = (1, 0, 0)$.

Tasks

Task 1. Define symbolic logits z_1, z_2, z_3 and softmax probabilities p_1, p_2, p_3.

Task 2. Build the negative log-likelihood $L = -\log p_1$ for the observed target token.

Task 3. Differentiate L with respect to z_1, z_2, z_3.

Task 4. Verify symbolically that the gradients match the pattern $p - e_1$.

Task 5. Define temperature-scaled probabilities $p_k(\tau)$.

Task 6. Substitute $z = (2, 1, 0)$ and evaluate probabilities for $\tau = 1$, $\tau = 1/2$, and $\tau = 2$.

Task 7. Compute the entropy of the exact distribution $(1/2, 1/4, 1/4)$.

Task 8. Define perplexity as $\exp(L)$, where L is an average negative log-likelihood, and evaluate it at $L = 0.7$, 1.5, and 3.0.

Task 9. Compute a masked average next-token loss for a two-example padded batch.

Task 10. Compute renormalized top-2 and nucleus-sampling distributions for a sorted probability vector.

Task 11. Compute beam-search log scores and length-normalized scores for tiny candidate continuations.

Task 12. Write a short interpretation connecting the symbolic gradient $p - e_1$ to next-token training.

Task 13. Write a short interpretation explaining how temperature, filtering, and beam scoring change decoding without changing model parameters.